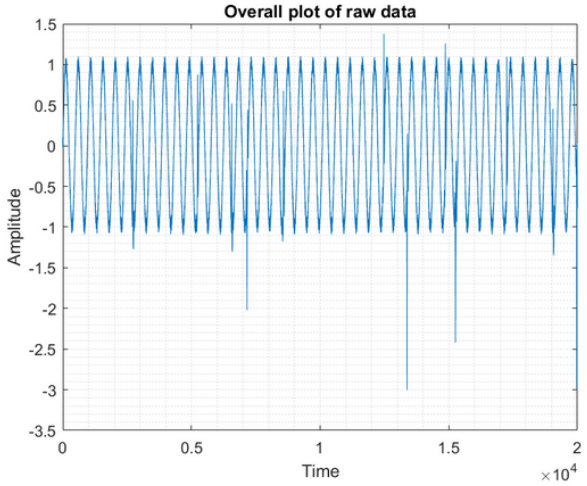
[redacted]

1. Section 1. Spike Sorting

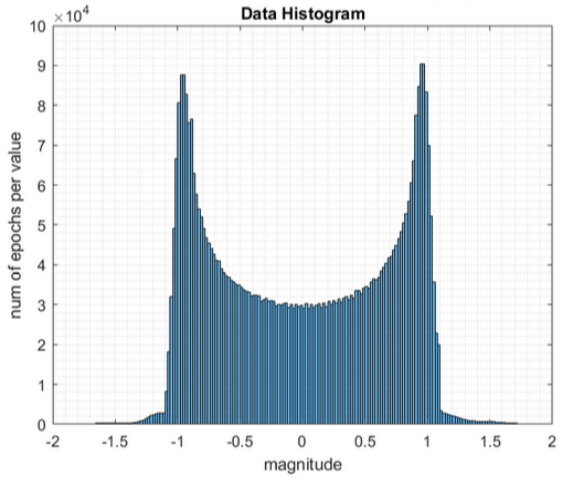
Through this assignment we will be going through the analysis of neuronal data in multiple stages. The First section is dedicated to spike sorting. Spike sorting is the process of measuring the neuronal spike of the target neighborhood of the electrode through a mathematical process consisting of filtering, spike detection, feature extraction, and clustering of spikes.

1. 1. Getting Started:

a) Data Plot: The below diagram plots amplitude of each recording against the time epoch of recording.

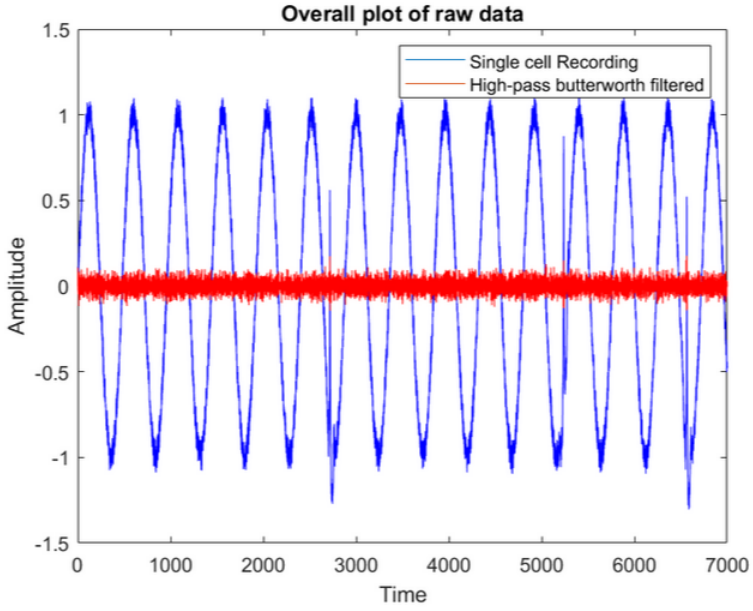


b) Data Histogram: This histogram shows the distribution of different recording values. As it can be seen in the given figure, the distribution peaks similarly around the voltage magnitude of 1mv and -1mv with a unitary step form in between the two peaks.



1. 2. Filtering the Data:

In this part we went on to design a high-pass Butterworth filter of the 7th order with a cutoff frequency of 300Hz, and applying it to the data. The figure below depicts the filtered data when shown over the raw data.

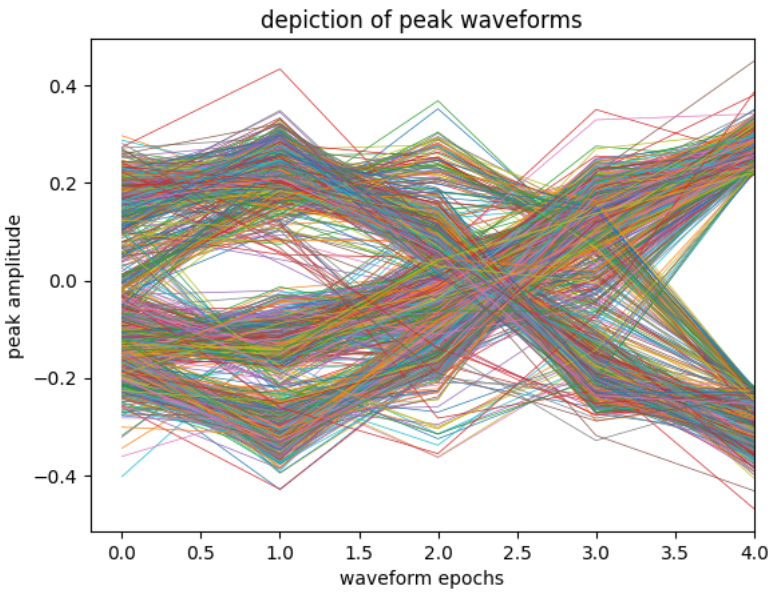


1. 3. Detecting the Spike:

In this part we first calculated the voltage threshold with the given equation which yielded the value of [0.2165574093077027]. Then by using the difference of the main signal with the threshold of the butterworth filtered signal.

For the aim of peak detection, as per mentioned above, the suggested algorithm was applied and two arrays of the peak point values and their indexes/locations in the dataset were saved separately. Then in order to extract the waveforms of each peak, we saved the peak point info and each of their two coresiding neighbors into a 5-column matrix.

Lastly, as per requested in the project manual, we depicted all the graphs of the peaks (or better say the regarding plot of each row of the matrix,) onto the graph given below. All peak points have the same general shape with the rises and falls being almost identical, though the peak amplitudes vary among samples and also the peaks can either be on the negative amplitude spectrum or the positive.



1. 4. Extracting Features:

For the purpose of feature extraction, we employ sklearn's features here to apply Principal Component Analysis on the waveforms. And then move on to choosing the most informative principal components by comparing PCA scores.

In general Principal Component Analysis, or PCA, is a dimensionality reduction method that is often used to reduce the dimensionality of large data sets, by transforming a large set of variables into a smaller one that still contains most of the information in the large set. So the goal is to choose features or variables of the dataset which can preserve as much information as possible.

The used gadgets actually pass the matrix of data through a covariance matrix of all variables and the yielded components or covariances which are the highest are then chosen as the main or principal components of analysis.

Here we retain from plotting any data but by using the mentioned tools, the dimensionality has been reduced and the three PCAs with the most covariance among the data have been chosen for the next part.

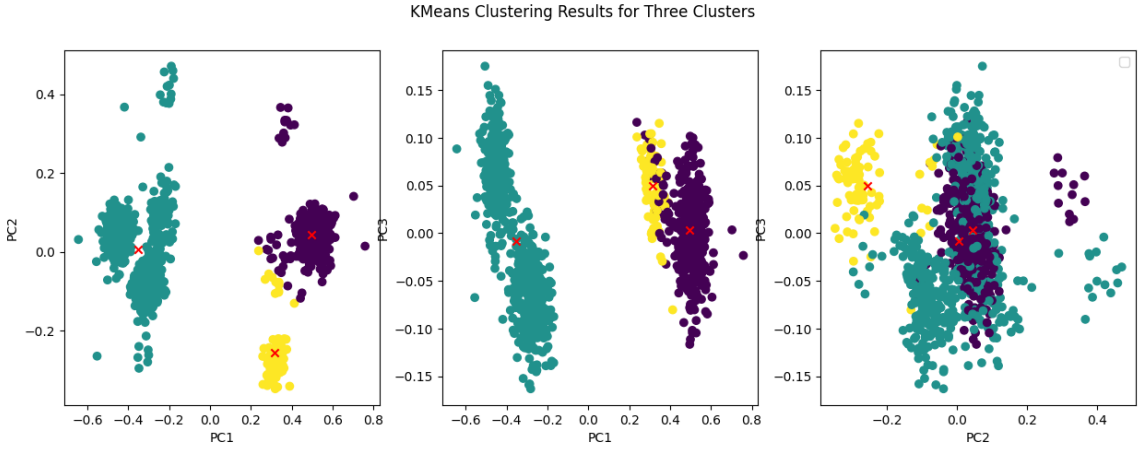
```
pca = PCA(n_components = 3)
pca_features = pca.fit_transform(wf)

print('Shape before PCA: ', wf.shape)
print('Shape after PCA: ', pca_features.shape)
```

```
Shape before PCA:  (1660, 5)
Shape after PCA:  (1660, 3)
```

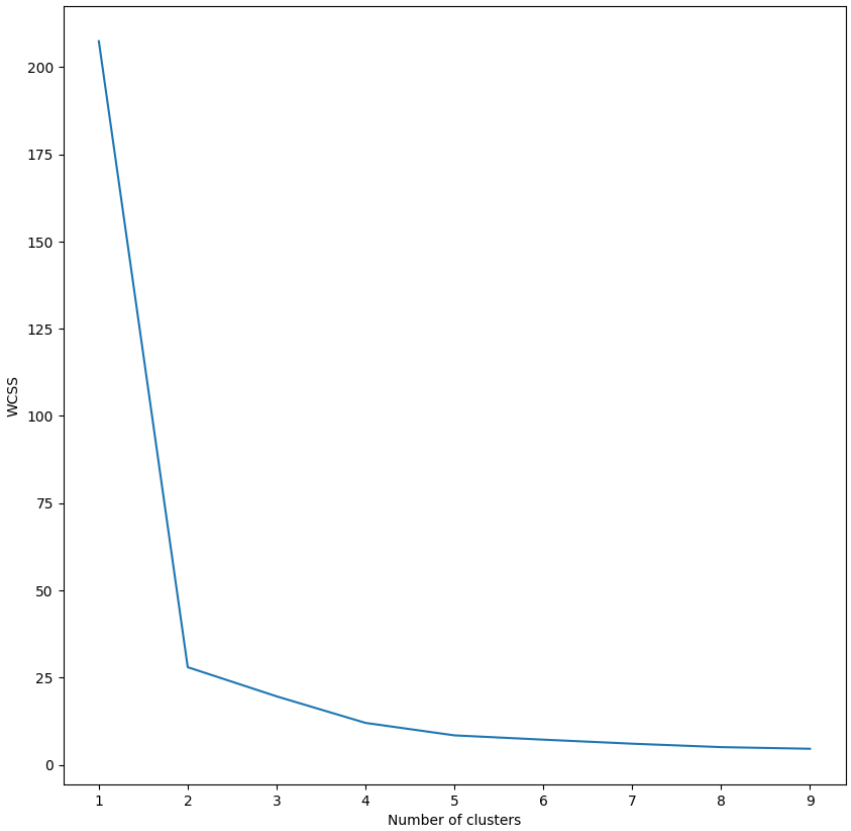
1. 5. Clustering the Spikes:

In this part we apply a Kmeans clustering algorithm to the waveform matrix which was previously passed through a PCA analysis. The three clusters of data relate directly to the three chosen PCAs of the previous part.



Now based on the three new PCAs or the three chosen features of analysis we train the model through the KMeans clustering algorithm to form clusters of the data, in regards to the chosen variables, around cluster centroids, which are depicted as red crosses on each plot. These centroids are the arithmetic mean of all the points belonging to a cluster. Each subplot analyzes the clusters correlated with a pair of PCAs.

For the next part, instead of manually changing the K value to find the best number of clusters, we will apply the *elbow method.* This method can be achieved by using **WCSS** or the sum of squares of distances of datapoints. As it can be seen in the figure below, the elbow_which yields the best clustering option, is around 4.

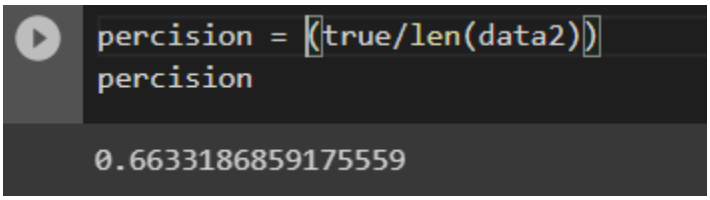


For the sake of checking the results of the elbow method, we repeat the previous plottings through two clusters. As it can be seen, the centers and thus the clusters are positioned further apart than the previous figure. Also it is notable that considering the first and second PCAs would be the best course of action since apparently the second and third



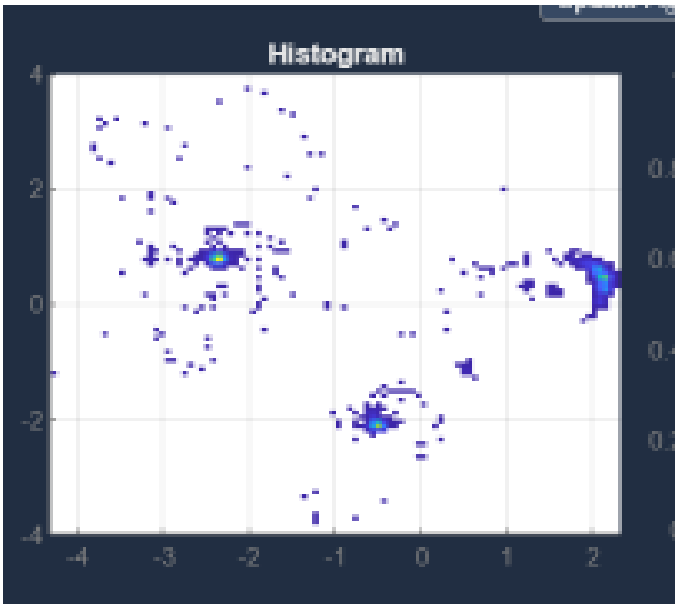
1. 6. Analyzing Spikes.mat:

The algorithm implemented in this part is basically a counter set to compare each element of *spikes.mat* with each saved index (which is the time of each detected peak, saved in a separate array). Then the precision is calculated by dividing the number of true detections by the number of actual action potentials.
The yielded precision percentage by applying the above algorithm is 66.332 which is unfortunately not a very good precision which could be due to the noisiness of the initial signal recording or the wrong estimations of the voltage threshold.



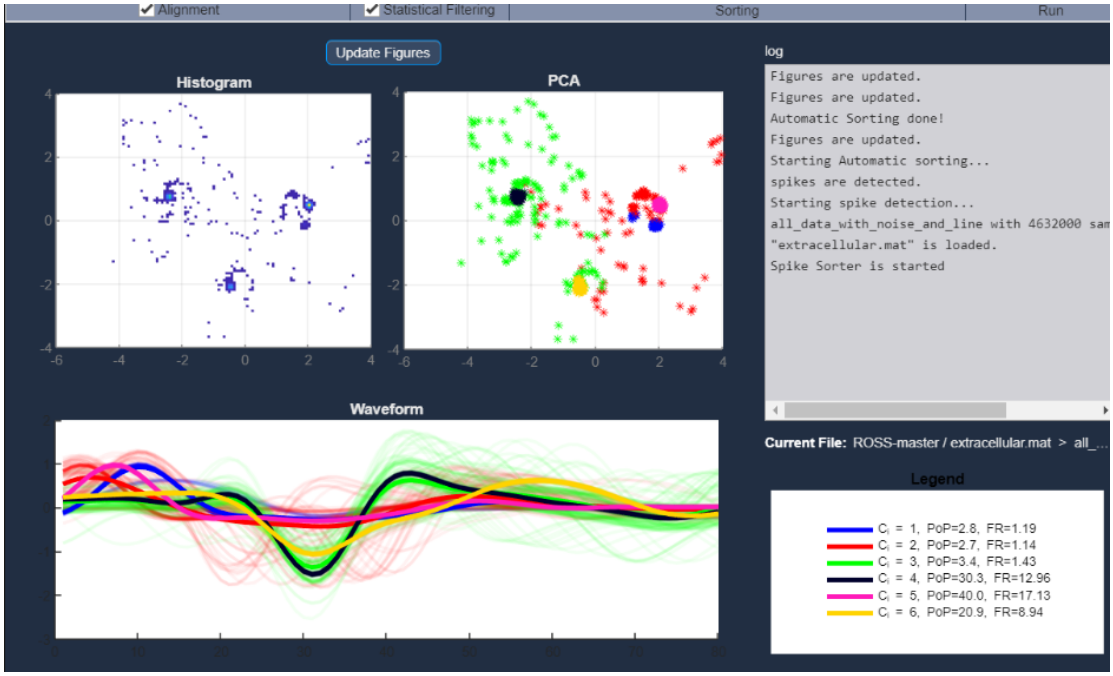
2. Section 2. Spike Sorting with ROSS

In this section we use the ROSS application through the MATLAB environment.
First we attempt to detect the peaks of the raw data by feeding it through the application. For filtering the data we use the built-in settings of this app and set the filter to a median based threshold plus a highpass butterworth filter of order 7 with a cut-off range of 300 to 3000 Hz.
The detected peaks are depicted below.



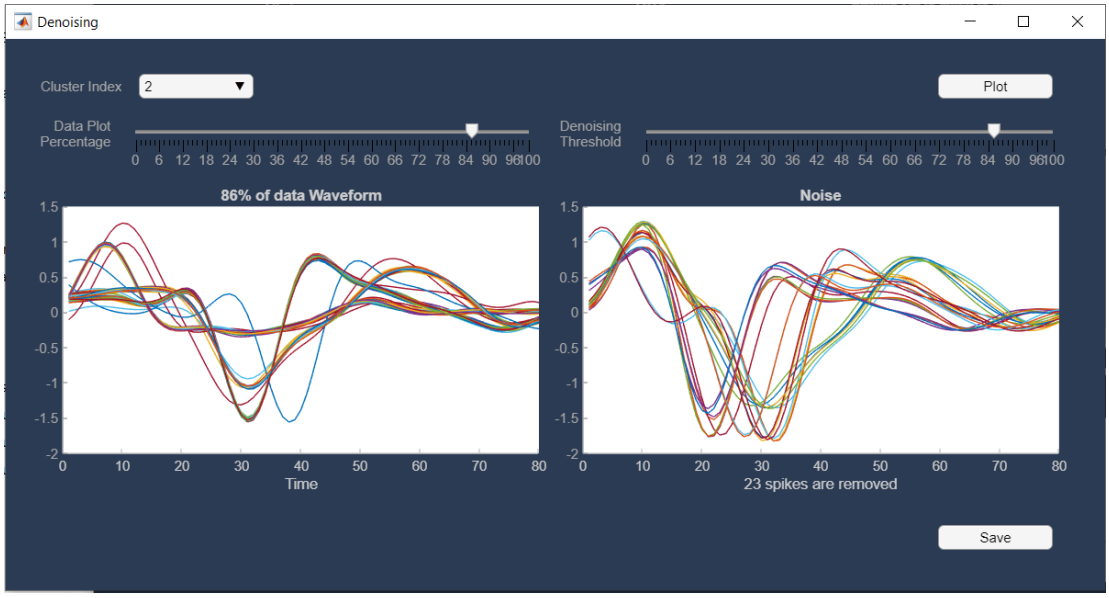
As it can be seen in the initial histogram of the raw data, the data will most efficiently be divided into three clusters where each cluster is made up of data points with the most similar features.

Then, per instructions of the project manual, we move on to inspecting the auto sorting options of the application. By using this feature, the data points' features were divided into six main PCAs where three of which are the most informative ones yielding the best division factor among all data. As it can be seen in the given figures, the three PCAs color coded with pink, black, and yellow can showcase a better distinctive division of the data.



In the next part of this question, we were asked to analyze the manual sorting of the application.

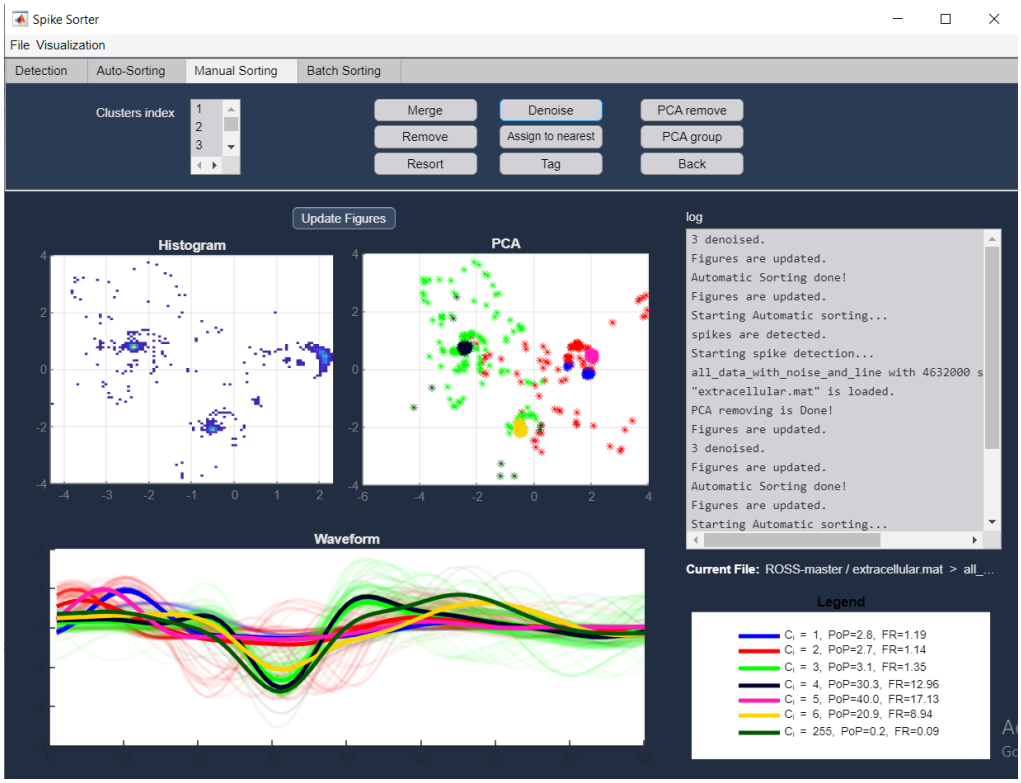
Through this tab, we can manually adjust the PCAs of interest or the distinct clusters of data which are being analyzed. As an initial step, here we set a denoising factor of 85% as per instructed. The general plot of data with a chosen cluster index of two and three can be seen below.

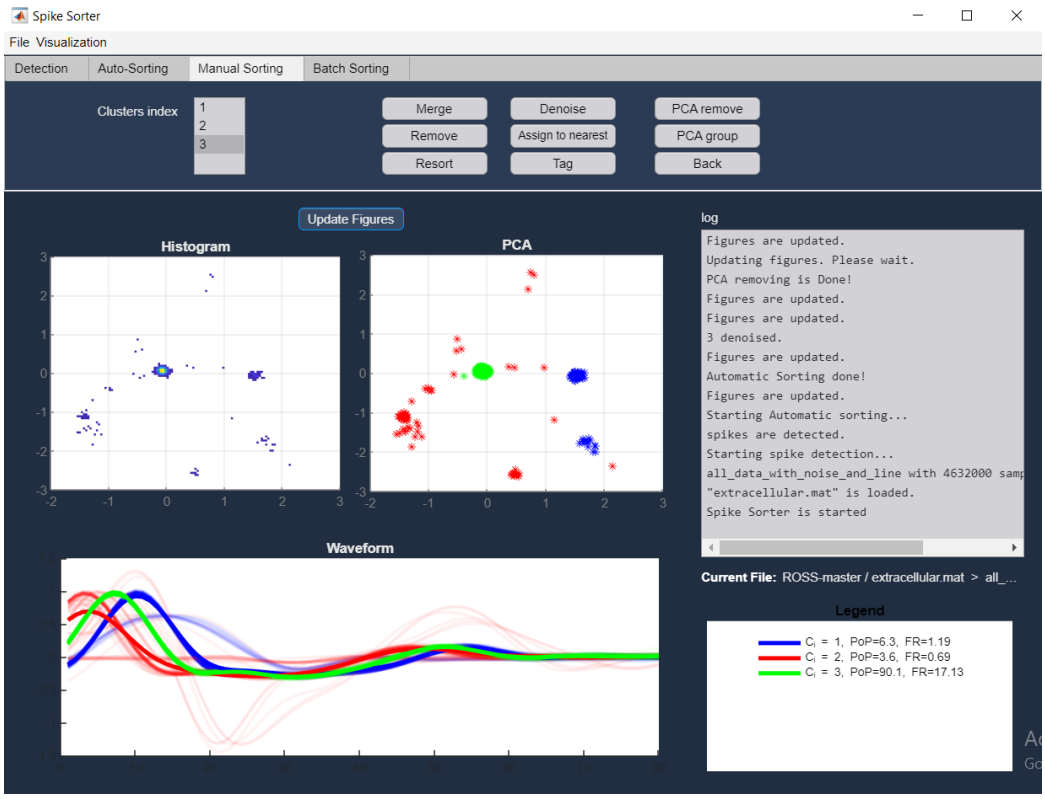




After denoising the signal, we attempt to remove PCAs by manually choosing certain clusters of data to sort. Here we can see an example of different clusters being studied through a 3 cluster division.

In the pictures below, we can see the PCA plots before and after removing PCAs, or choosing a specific cluster in other words. It can be seen that by removing some principal components and focusing on a certain group of data points we can find more informative PCAs to sort peaks.

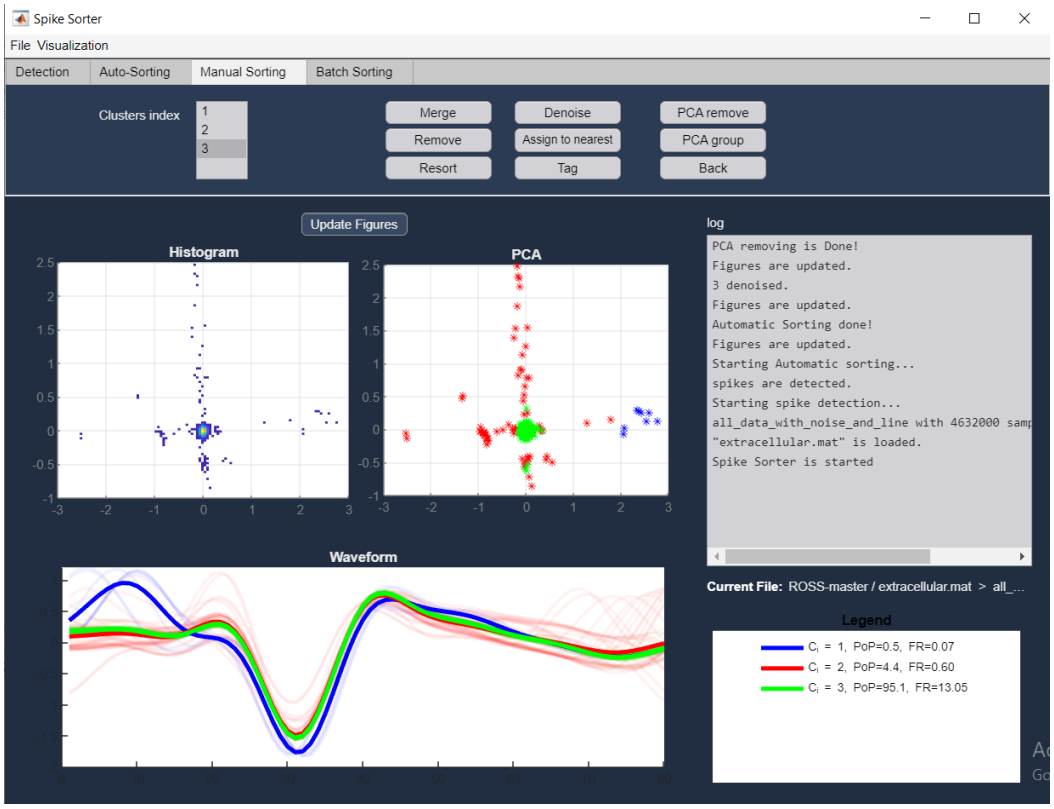


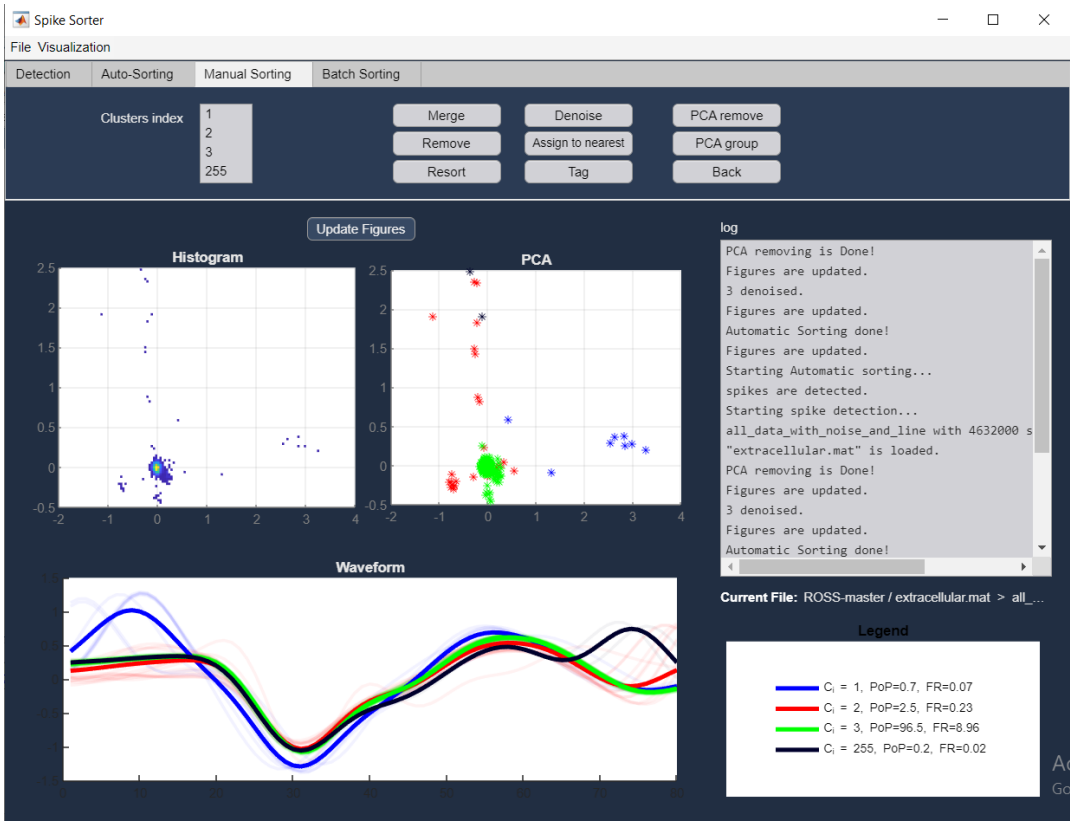


By applying auto-sorting to the chosen clusters and removing PCAs and denoising the signal, we see that the chosen cluster was not divided into smaller clusters which can be a sign of precise division of the data points and PCAs.

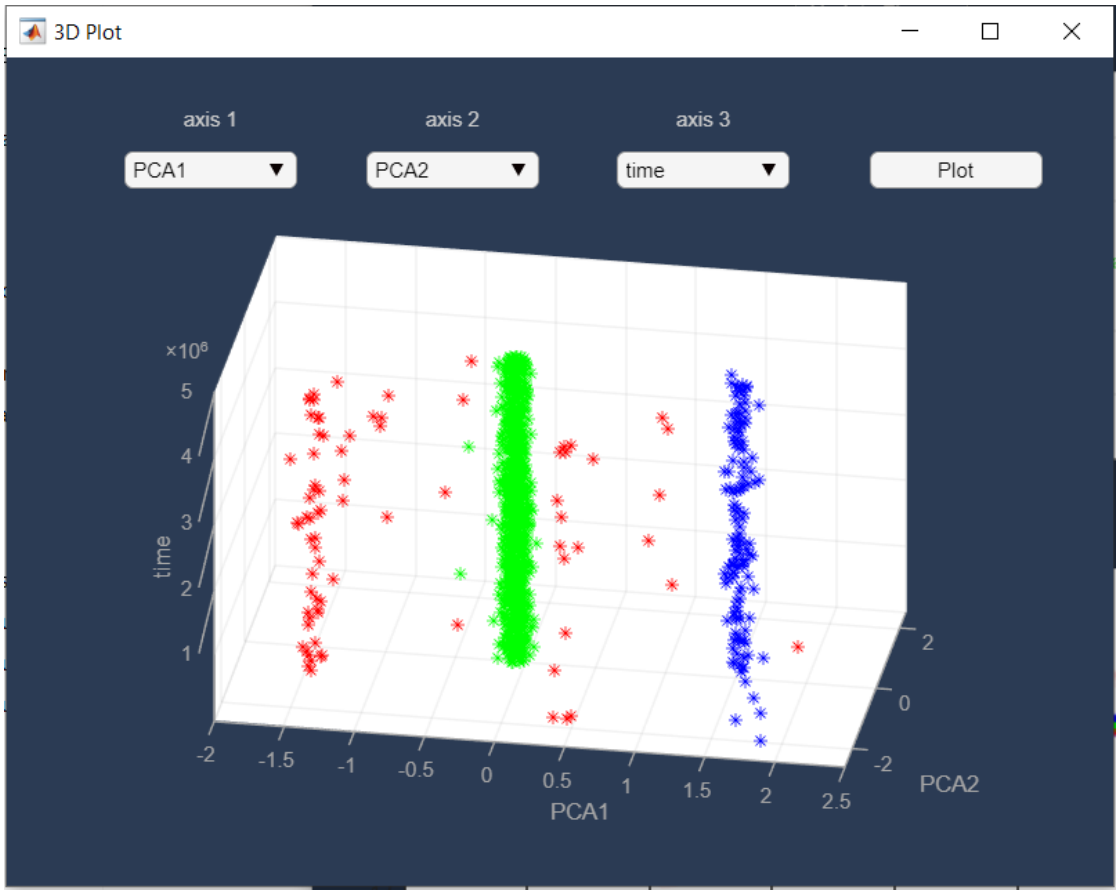
Next we moved on to studying the visualization capabilities of the application.

First we study the waveform of the two remaining clusters besides the one chosen for the above figure.

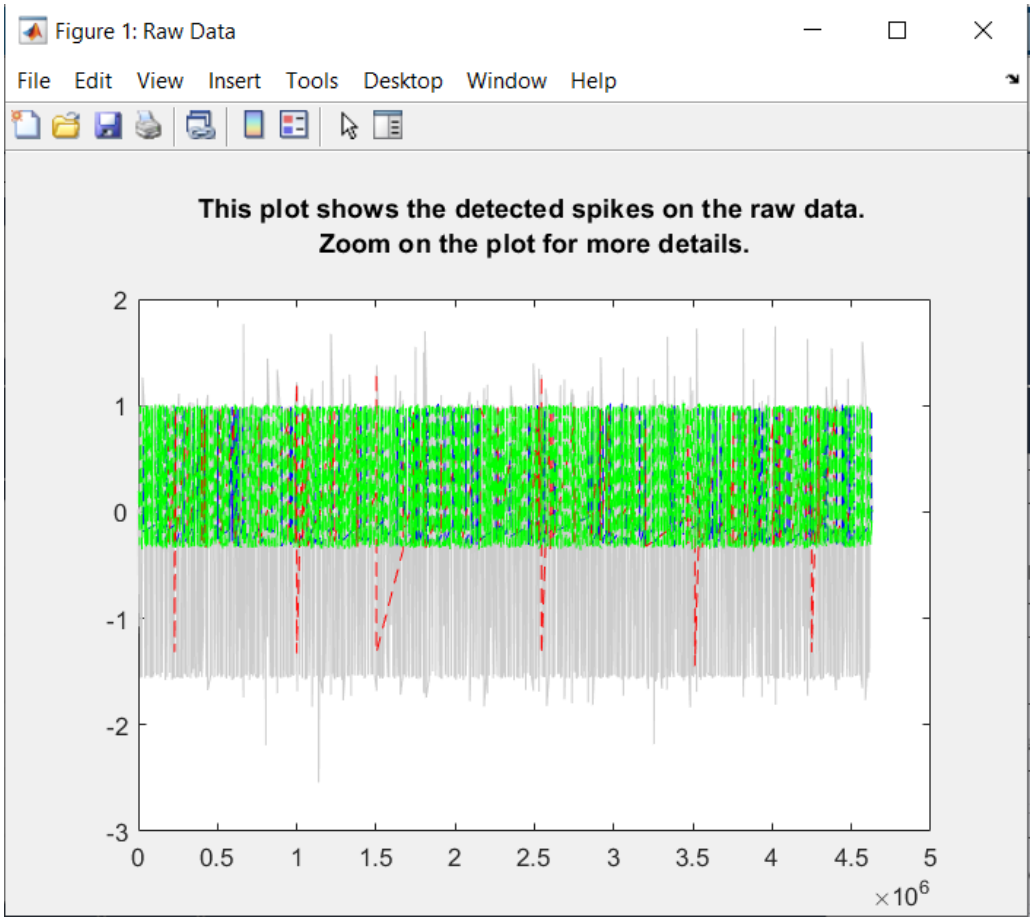

Spike Sorter
File Visualization
Detection
Auto-Sorting
Manual Sorting
Batch Sorting
Clusters index
1
2
3
Merge
Remove
Resort
Denoise
Assign to nearest
Tag
PCA remove
PCA group
Back
Update Figures
Histogram
PCA
Waveform
log
PCA removing is Done!
Figures are updated.
3 denoised.
Figures are updated.
Automatic Sorting done!
Figures are updated.
Starting Automatic sorting...
spikes are detected.
Starting spike detection...
all_data_with_noise_and_line with 4632000 samp
"extracellular.mat" is loaded.
Spike Sorter is started
Current File: ROSS-master / extracellular.mat > all_...
Legend
Cl = 1, PoP=0.5, FR=0.07
Cl = 2, PoP=4.4, FR=0.60
Cl = 3, PoP=95.1, FR=13.05


Spike Sorter
File Visualization
Detection
Auto-Sorting
Manual Sorting
Batch Sorting
Clusters index
1
2
3
255
Merge
Remove
Resort
Denoise
Assign to nearest
Tag
PCA remove
PCA group
Back
Update Figures
Histogram
PCA
Waveform
log
PCA removing is Done!
Figures are updated.
3 denoised.
Figures are updated.
Automatic Sorting done!
Figures are updated.
Starting Automatic sorting...
spikes are detected.
Starting spike detection...
all_data_with_noise_and_line with 4632000 s
"extracellular.mat" is loaded.
PCA removing is Done!
Figures are updated.
3 denoised.
Figures are updated.
Automatic Sorting done!
Current File: ROSS-master / extracellular.mat > all_...
Legend
Cl = 1, PoP=0.7, FR=0.07
Cl = 2, PoP=2.5, FR=0.23
Cl = 3, PoP=96.5, FR=8.96
Cl = 255, PoP=0.2, FR=0.02

As it can be seen, the first two chosen clusters were more concentrated into a center with less PCAs being able to differentiate them from the rest of the data, and the second figure being the best choice of focus between the two.
However the third chosen cluster was much less concentrated thus needing more principal components to annotate its features in a distinctive manner.
Then we attempt to plot a 3d form of the PCA plots with two axes being the two best PCAs and the other axis posing as a time metric.



And also a general plot of the raw data with its peaks being highlighted, which is not that informative due to the large size of the dataset but can still show some features or general overview of the data.



3. Section 3. Analysis of Single Neuron Activity

Throughout this part, we will be analyzing the information processing capabilities of a single neuron cell through d-prime and mutual information metrics.
The given data is a 3d matrix of 77 x 40 x 900 which represents number of stimuli x number of neurons x time samples. This dataset is accompanied by an array of length 77 which represent the label

3. 1. Visualization:

For this part we start by designing a moving window to average the data along the time axis. I decided to design a convolution window to complete this task. Then we proceed by passing the data through the filter and rewriting the data matrix from a dimension of (77x40x900) to (77x40x51). So as it can be seen the averaging window did not only denoise the data but it also lowered the dimensionality of the matrix.
Here we have stratified the data based on its labels (or rows) and have plotted all of the data belonging to each strat in one plot. Meaning that each subplot of the

below figure shows the all neuronal data belonging to all samples of a strat in regards to its time, in both the filtered and raw data.

